Algoritmi paraleli si distribuiți

Modele de calcul paralel

+ C++ Threads

Paralelism implicit si explicit

- Greșeală frecventă: Programele paralele sunt dificil de scris în comparație cu cele secvențiale.
  - Totuși, gândirea/abordarea paralelă/distribuită trebuie antrenată și înțeleasă în detaliu cu avantajele și dezavantajele specifice.
  - ***Limitare majoră***: depanarea comportamentului nedefinit (ex., nedeterminist) care poate apărea mult mai frecvent în programele paralele.
- Paralelism implicit
  - Programatorul NU specifică în mod clar modul de paralelizare. Paralelizarea este realizată automat de compilator sau la rulare.
- Paralelism explicit [*mult mai util/important pentru programatori*]
  - Paralelismul este specifcat în codul sursă prin proiectare, directive complexe sau utilizarea de biblioteci/sisteme specifice.

Paralelism implicit

- Care din urmatoarele instrucțiuni se pot executa în paralel?

```
a = 2; b = 3;
```

Da, cele două instrucțiuni nu depind una de alta.

```
a = 3; b = a;
```

Nu, execuția paralelă conduce la nedeterminism sau comportament nedefinit.

```
for (i=0; i<100; i++)
    b[i] = i;
```

Da, nici o iterație nu depinde de alta.

```
a = f(x); b = g(x);
```

Nu putem spune fără să cunoaștem funcțiile *f* și *g*.
Se poate întâmpla ca instrucțini din *f* și *g* să modifice aceleași variabile.

Paralelism explicit: nivele

- Nivelul hardware
  - Prin sistemul și organizarea sa fizică.
  - Prin rețeaua de comunicare.
- Organizarea logică
  - Modul în care programatorul vede platforma pe care va rula aplicația paralelă.
- Maparea proceselor pe procesoare
  - Acesta este nivelul de execuție.

Modelul PRAM

- Este un model teoretic care permite schimbul de date între procesoare în timp constant (i.e., fără cost).
- Evident, o mașină PRAM nu se poate construi fizic
  - Costul de conectare a celor **p** procesoare la o memorie de dimensiune **m** astfel încât accesele la memorie să nu interfereze este **O(p*m)**, ceea ce reprezintă o valoare foarte mare pentru valorile lui ***m*** din practică.
- Măsuri ale complexității
  - Pentru fiecare procesor
    - Timpul: numărul de instrucțiuni executate
    - Spațiu: numarul de zone de memorie accesate
  - Pentru mașina PRAM
    - Timpul: timpul maxim de rulare dintre timpii procesoarelor
    - Paralelismul: numărul maxim de procesoare active

Modelul PRAM

- Numarul foarte mare de conexiuni ar conduce la dificultatea realizării sincronizărilor necesare.
- Este cel mai bun model conceptual pentru proiectarea algoritmilor paraleli eficienți.

Instrucțiune simplă paralelă

For all x in X do in parallel
 # instruction (x)

→

Pentru fiecare *x*, PRAM va desemna un procesor care va executa *# instruction (x)*

Categorii de PRAM [în funcție de accesul la memoria partajată]

- Restricții pot fi impuse pentru citirea/scrierea simultană în memoria partajată
- În funcție de modul în care este administrat accesul simultan avem:
  - **E**xclusive Read, **E**xclusive Write - EREW PRAM
  - **C**oncurrent Read, **E**xclusive Write - CREW PRAM
  - **E**xclusive Read, **C**oncurrent Write - ERCW PRAM (pentru completitudine)
  - **C**oncurrent Read, **C**oncurrent Write - CRCW PRAM

Soluționarea scrierii concurente

- Existența citirilor concurente nu crează probleme în logica programelor.
- Scrierea concurentă a două sau mai multe procese în aceeași zonă de memorie necesită sincronizare/arbitraj; printre metodele de rezolvare a acestei probleme avem:
  - Valoare comună : toate procesele trebuie să scrie aceeași valoare.
  - Arbitrar: scrierile sunt efectuate arbitrar.
  - Prioritizat: procesul cu prioritate mai mare urmează.
  - Sumă: se scrie suma valorilor trimise de procese.

EREW => CREW =>Valoare comună => Arbitrar => Prioritizat

Scenariu cel mai realist ……..> Scenariu cel mai **<u>puțin</u>** realist

PRAM: Exemplul 1. Determinarea sumei prefixelor

- **Input**: o mulțime ordonată de elemente $A = \{a_0, a_1, \ldots, a_{n-1}\}$
- **Output**: mulțimea ordonată $\{a_0, (a_0 + a_1), \ldots, (a_0 + a_1 + \ldots + a_{n-1})\}$
- Exemplu:
  - Input: {4, 3, 5, 1, 8, 11, 2, 9}
  - Output: {4, 7, 12, 13, 21, 32, 34, 43}
  - Rezolvarea secvențială necesită **O(n)**

PRAM: Exemplul 1. Determinarea paralelă a sumei prefixelor

- Soluția secvențială care folosește arbori binari
  - Pentru fiecare nod intern calculam valoarea ca fiind suma valorilor tuturor frunzelor subarborelul corespunzător.
    - Abordarea este bottom-up.
    - Formularea recursivă este sum[v] = sum [left[v]] + sum [right[v]]
    - Putem folosi un array/vector pentru valori. Care este complexitatea?

43

13 30

7 6 19 11

4 3 5 1 8 11 2 9

PRAM: Exemplul 1. Determinarea paralelă a sumei prefixelor

- Abordare paralelă:

4	3	5	1	8	11	2	9
7 (p0)		6 (p1)		19 (p2)		11 (p3)	
13 (p0)				30 (p2)			
43 (p0)							

Abordarea paralelă rezolvă problema în $O(log_2(n))$

Calcul în timp constant O(1)

- Arhimede: *”Give me a lever long enough and a place to stand and I will move the earth.”*

- În zilele noastre ... Dacă am avea la dispoziție o mașină paralelă cu suficent de multe procesoare am putea determina valoarea cea mai mică în timp constant.

Provocările algoritmilor paraleli

- Paralelizarea soluțiilor secvențiale reprezintă o provocare algoritmică.
  - Scop: Dezvoltarea gândirii paralele!

- *Exemplu*: Determinarea sumei a ***n*** numere.
  - Cum influențează numarul de procesoare algoritmul paralel?
  - Cum influențează costul comunicării soluția paralelă?

Modelele algoritmice

- Oferă o bază comună pentru dezvoltarea și compararea algoritmilor paraleli.
- In general, folosesc modelul arhitectural al unei mașini paralele (i.e., multiprocesor) cu memorie partajată.
- Memoria partajată reprezintă o abstractizare utilă din punctul de vedere al programatorului, mai ales în fazele inițiale ale proeictării soluției.

Modelul cu paralelizare a datelor

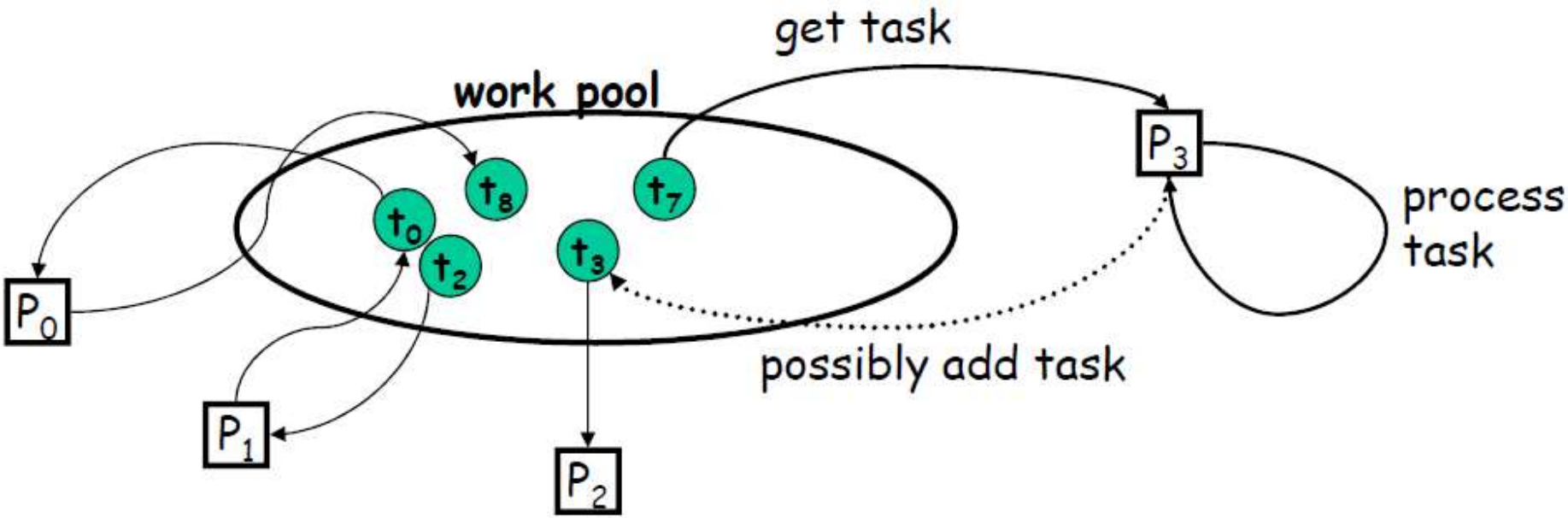
- Principiu
  - Datele de intrare sunt împărțite între procesoare.
  - Se procesează în paralel segmente de date diferite.
  - Comunicarea se rezumă la informații marginale.
- Proprietăți
  - Include paralelismul în buclă (en., loop paralelism).
  - Foarte adecvat pentru mașinile SIMD.
  - Comunicația este de obicei implicită (i.e., nu necesită apeluri specifice).

Modelul de paralelizare cu grafuri

- Se descompune algoritmul în mai multe secțiuni distincte.
- Secțiunile se atribuie la procesoare diferite.
- Se folosește *fork() / join() / spawn()*
  - *fork()* : Crează un nou fir, denumit fir copil.
  - *join()* : Întrerupe firul curent până când alt fir își termină execuția.
  - *spawn()*: *Crează și execută un nou fir.*
- De obicei, nu conduce la un nivel/grad înalt de paralelizare
  - Ajută doar la executarea task-urilor în ordinea corectă.

Modelul de paralelizare bazat pe *pool*

- Asignarea task-urilor este realizată dynamic.
- De obicei, fiecare task are asociată o cantitate mică de date.
- Pool-ul de task-uri (ex., coadă de prioritate, tabelă de dispersie, arbore) poate fi centralizat sau distribuit.



Algoritm paralel vs. Formulare paralelă

- Formularea paralelă
  - Se referă la paralelizarea algoritmului secvențial.
- Algoritmul paralel
  - Poate fi un algoritm complet diferit de cel secvențial.
- Scopul este să definim corect formulări paralele
  - Putem avea algoritmi paraleli care nu sunt obținuți pe baza celor secvențiali.
- Algorim/formulare paralelă
  - Task = procesare care poate fi efectuată concurent/independent.
  - Procese vs. Procesoare = asignarea task-urilor pe procesoare
    - Distribuția I/O și a rezultatelor intermediare pe procesoare diferite
  - Administrarea accesului la datele partajate (input sau intermediare)
  - Sincronizarea procesoarelor la diferite momente ale execuției paralele
- **Scop**: Maximizarea concurenței și reducerea costurilor suplimentare datorate paralelizării => Potențial maxim de accelerare.

Procese vs. Procesoare

- Process = Cod/aplicație care execută un task.
- Procesor = HW care execută fizic calculele.
- De obicei, exista o corespondență 1:1 între procese și procesoare.
- Pentru a obține o accelerare față de versiunea secvențială, programul paralel trebuie să aibă mai multe procese active simultan, care să rezolve task-uri distincte.

Determinarea procesărilor care se pot desfășura concurent

- Descompunerea
  - Este procesul prin care se împarte toată procesarea necesară în bucăți mai mici, adică **task**-uri.
- Task-urile sunt definite de programator si sunt considerate atomice (i.e., indivizibile).
- Granularitatea unui task se referă la dimensiunea acestuia
  - Ex. A[NxN] x b[Nx1] = y [Nx1]
    - Granularitate mică => N task-uri, cate unul pentru fiecare element din y
    - Granularitate medie => N/3 task-uri, fiecare task procesează 3 elemente din y

Graful de dependințe între task-uri

- Procesarea este văzută ca un graf direcționat aciclic (DAG).
- Fiecare nod din graf este un task.
- O muchie din graf reprezintă o relație de dependență.
  - **A->B** se interpretează ca task-ul A trebuie să se execute înainte de task-ul B, sau task-ul B necesită terminarea task-ului A.

Metode de descompunere

- Descompunerea datelor
- Descompunerea task-urilor
  - Descompunerea recursivă
    - Pentru problemele care folosesc abordarea *divide et impera*
    - Fiecare din subproblemele generate de pasul *divide* devine un task.
    - Care este concurența medie? Care este lungimea căii critice?
    - *Exemple*: quicksort, determinarea minimului, etc.
  - Descompunerea exploratorie
  - Descompunerea speculativă
- Descompunerea hibridă

Descompunerea recursivă

- Cât de bune sunt descompunerile pe care le realizează?
  - Care este concurența medie?
  - Care este calea critică?

- Cum putem măsura calitatea descompunerii pentru *quicksort* sau determinarea minimului?

Descompunerea datelor

- Se folosește pentru obținerea concurenței la problemele care au ca input o cantitate mare de date.
- Ideea este să definim task-urile având în vedere cantitatea de date.
- Descompunerea se realizează în 2 pași:
  - Partiționarea datelor
  - Realizarea partiționării procesării pe baza partiționării datelor
- Ce date ar trebui să partiționăm?
  - Input/output/rezultate intermediare?
- Cum obținem/definim task-urile necesare?
  - Regula este că task-ul care deține datele realizează procesarea.

Exemplu: Înmulțirea a două matrici

- Partiționarea datelor de ieșire

A[2x2] x B[2x2] = C[2x2]

- Task 1: C11 = A11*B11 + A12*B21
- Task 2: C12 = A12*B12 + A12*B22
- Task 3: C21 = A21*B11 + A22*B21
- Task 4: C22 = A22*B12 + A22*B22

Descompunerea datelor și descompunerea exploratorie

Descompunerea datelor: Este cea mai folosită tehnică

- Procesarea paralelă se aplică în multe situații în care dimensiunea datelor de intrare este mare.
- Împărțirea procesării be baza datelor este un mod natural de a obține un grad ridicat de concurență.

Descompunerea exploratorie

- Se folosește atunci când descompunem/împărțim spațiul soluțiilor în mulțimi disjuncte și pentru fiecare mutime creăm câte un Task.
- Poate fi folosită pentru probleme specifice (ex., problema puzzel-ului cu 15 piese, https://en.wikipedia.org/wiki/15_puzzle)
- Poate prezenta anomalii ale accelerării: accelerare supraliniară

Descompunerea speculativă

- Se folosește pentru a obține concurența atunci când următorul *pas* este unul din multe posibilități și se poate determina numai după ce task-ul curent se termină.
- Acest tip de descompunere *presupune* un anumit rezultat/răspuns al task-ului curent și *execută* câțiva din pașii următori.
  - Mecanismul este asemănător cu execuția speculativă la nivel de microprocesor.
- Dacă predicțiile sunt greșite
  - Rezultatele execuției speculative devin inutile și sunt pierdute/aruncate
- Totuși, se poate întâmpla ca aceasta să fie singura modalitate de a obține concurența.

Alocarea task-urilor

- De ce este important modul în care alocăm task-urile?
  - Nu putem să alocăm aleator tak-urile procesoarelor disponibile?
- Alocarea este o operație critică ce trebuie să minimizeze costurile suplimentare (en., *overhead*) care apar în urma paralelizării.
  - To = p*Tp + Ts // Ts = timpul de execuție serială, Tp este timpul de execuție // paralelă pe p procesoare
  - Surse ale costurilor suplimentare
    - Încărcarea dezechilibrată a procesoarelor
    - Comunicarea între procese
      - Coordonare/sincronizare/partajarea datelor

Alocarea task-urilor

- Alocările eficiente încearcă să:
  - Maximizeze concurența prin alocarea de task-uri independente la procesoare diferite.
  - Minimizeze tipul total de execuție prin asigurarea faptului că task-urile care se găsesc pe ***calea critică*** sunt cele care sunt executate cât mai devreme posibil.
    - Cum putem determina calea critică?
  - Aloce task-urile care au grad mare de interacține între ele aceluiași proces.
    - Cum putem determina mulțimile de task-uri care au grad mare de interacțiune între ele?
      - Ce algoritmi din teoria grafurilor (nod – task; muchie = interacțiune) putem folosi?

Dificultatea alocării task-urilor

- Trebuie avute în vedere dependințele dintre task-uri și graful de interacțiuni/dependințe dintre task-uri
  - Sunt task-urile disponibile apriori?
    - Crearea/generarea task-urior poate fi statică sau dinamică.
  - Care sunt cerințele task-urilor în ceea ce privește necesitățile de calcul?
    - Sunt ele uniforme? ... Aceleași pentru toate task-urile?
    - Sunt ele cunoscute apriori?
  - Ce cantitate de date este asociată cu fiecare task?
  - Care sunt șabloanele de intaracțiune dintre task-uri?
    - Statice sau dinamice?
    - Le cunoaștem apriori?
    - Depind de datele prelucrate?
    - Sunt read-only sau read-write?

Tehnici de alocare pentru echilibrarea încărcării

- Alocare statică
  - Task-urile sunt alocate procesoarelor înainte de execuție.
  - Se aplică task-urilor care sunt generate static sau au necesități de calcul cunoscute sau uniforme.
  - Este potrivită pentru algoritmii care folosesc descompunerea datelor sau prelucrează date (input/date intermediare/output) sub forma tablourilor.
- Alocare dinamică
  - Task-urile sunt distribuite procesoarelor în timul execuției programului
  - Se aplică task-urilor care
    - Sunt generate dinamic
    - Nu li se conoaște apriori necesitatea de calcul

Metode de echilibrare dinamică a încărcării

- Metode centralizate
  - Anumite procesoare sunt responsabile pentru procesări
    - Modelul client-server
- Metode distribuite
  - Încărcarea poate fi transferată de la un procesor la altul
  - Probleme
    - Cum determinăm perechile de procesoare care își deleagă încărcarea
    - Cine inițiază transferul? push vs.pull
    - Cat de multă încărcare este transferată?

Alocările trebuie să minimizeze costurile suplimentare generate de interacțiuni

- Maximizarea localizării datelor
- Minimizarea volumului de date schimbat
- Minimizarea frecvenței interacțiunilor
- Minimizarea disputelor
- Suprapunerea interacțiunilor cu efectuarea de calcule
- Replicarea selectivă a datelor și calculelor

Obs: Obținerea mecanismului optim se realizează iterativ și depinde de interacțiunea dintre descompunere și alocare.

Modele de calcul paralel

- Cu memorie partajată
- Cu fire de execuție
- Cu transmitere de mesaje
- Cu paralelizarea datelor
- Hibride

Aceste modele de calcul paralel sunt abstracte și nu iau în calcul arhitectura HW a sistemului. Teoretic, acestea pot fi implementate pe orice mașină fizică.

Alegerea modelului este de obicei o combinație între ceea ce avem la dispoziție ca HW si alegerea programatorului având în vedere problema propusă.

Modelul cu fire de execuție

- În acest model un singur proces poate avea mai multe căi de execuție concurentă.
- *Exemplu*: Un program principal care include mai multe subprograme.
  - Programul principal este lansat în execuție.
  - El crează mai multe task-uri (fire de execuție) care sunt programate să fie rulate de SO în mod concurent.
  - Fiecare fir are propriile date locale, dar partajează și rezursele programului principal. Alest lucru diminuează costurile suplimentare (en., overhead) cu replicarea resurselor programului principal pentru fiecare fir de execuție. Fiecare fir beneficiază de acces la memoria globală deoarece partajază spațiul de memorie al programului principal.

Modelul cu fire de execuție (cont.)

- Ceea ce procesează un fir poate fi înțeles ca un subprogram în cadrul programului principal. Orice fir poate executa orice subprogram în același timp cu celelalte fire de execuție.
- Firele comunică între ele cu ajutorul memoriei globale, pe care o modifică în mod concurent. Acest mode de lucru necesită sincronizare astfel încât niciodată sa nu avem două sau mai multe fire cu acces simultan la acceeasi zonă de memorie (i.e., variabilă).
- Firele se pot crea sau distruge, dar programul principal este cel care asigură resursele necesare până la terminarea aplicației.

- Firele de execuție sunt de obicei asociate cu arhitecturile cu memorie partajată și sistemele de operare.

Modelul cu fire de execuție (cont.)

- Din punct de vedere al implementării utilizarea firelor presupune:
  - O bibliotecă ce este utilizată de programul client paralel.
  - O mulțime de directive pentru compilator care sunt apelate în programul client.
  - ***Obs***: În ambele cazuri programarorul este cel care determină și administrează tot paralelismul.
- Implementările paralele cu fire de execuție sunt printre primele disponibile. Eforturile de standardizare s-au concretizat în două implementări ale firelor: POSIX Threads și OpenMP.

Modelul cu transmitere de mesaje

- Transmiterea de mesaje este printre cele mai utilizate modele de programare paralelă.
- Programele care folosesc transmiterea de mesaje:
  - Crează mai multe task-uri
  - Fiecare task are alocat un set de date
  - Fiecare task este unic identificat
  - Task-urile interacționeză prin trimiterea/primirea de mesaje către/de la alte task-uri.
  - Nu exclud crearea dinamică a task-urilor, execuția mai multor task-uri pe un singur procesor, sau execuția de programe diferite de către task-uri diferite.

Modelul cu transmitere de mesaje (cont.)

- In practică, de cele mai multe ori:
  - Programul crează un număr fix de task-uri identice la inițializare și nu permite crearea sau distrugerea de task-uri în timpul execuției.
  - Acest model implentează modelul SIMD (Single Program Multiple Data) deoarece fiecare task execută același cod (aceleași instrucțiuni) dar pe date diferite.
  - Mai multe task-uri se pot găsi pe aceeași mașină fizică, dar se pot afla și pe mașini diferite.
  - Transferul de date necesită ca fiecare proces să execute operații în mod coordonat cu altele. De exemplu, o operație *send* trebuie să aibă o operație pereche *receive* bine determinată. Sisteme mai avansate implementează *listener pattern* (vezi Inginerie software, șabloane de proiectare).

Modelul cu transmitere de mesaje (cont)

- Din punct de vedere al implementării trebuie utilizată o bibliotecă ce conține toate subprogramele care se folosesc în codul sursă. Programatorul este cel care stabilește întreg modul de paralelizare.
- MPI : lansat în 1994
- MPI-2 : lansat în 1996
- MPI 4.0 lansat în 2020

https://www.mpi-forum.org/

Modelul cu paralelizarea datelor

- Acest model valorifică o concurență de se obține și aplicarea aceleiași operații mai multor elemente dintr-o structură de date de tip tablou.
- O mulțime de task-uri vor lucra simultan și vor coopera pe aceeași structură de date, dar fiecare task va lucra pe o porțiune diferită de date față de celelalte task-uri.
- Exemple:
  - Adaugă 2 la toate elementele din tabloului
  - Crește salariul angajaților cu vechime de peste 5 ani
  - ***Obs***: Un program care implementează paralelizarea datelor constă dintr-o secvență de astfel de operații.

Modelul cu paralelizarea datelor (cont.)

- Compilatorul trebuie să fie informat de programator referitor la modul în care datele sunt distribuite către procesoare (i.e., cum sunt partiționate datele pe task-uri). Ulterior, compilatorul poate genera automat codul de comunicare.
- Pe sistemele cu memorie partajată, toate task-urile pot avea acces la structura de date prin memoria globală.
- Pe sistemele cu memorie distribuită, structura de date este împărțită și este disponibilă în bucăți (en., chunks) care se găsesc în memoria locală a fiecărui task.
  - Distribuția poate avea în vedere si replicarea redundantă.
- Implementările cu memorie distribuită se bazează pe compilator pentru a converti programul în cod cu apeluri de bibliotecă (ex., MPI) pentru a distribui datele către procese. Transferul de date este realizat transparent pentru programator.

Modelul cu memorie partajată

- În acest model, task-urile partajază un spațiu de adrese de memorie comun la care au acces (i.e., citire și scriere) în mod asincron.
- Mecanismele de tipul lacăte/semafoare trebuie folosite pentru gestionarea accesuli la memoria partajată.
  - Un avantaj pentru programator este faptul că un există noțiunea de *proprietar* al datelor. Acest aspect conduce la faptul ca nu este nevoie să specificăm explicit modul de cumunicare dintre producător și consumator.
- Acest model simplifică dezvoltarea aplicațiilor. Totuși, înțelegerea și administrarea valorilor datelor poate deveni dificilă pe cele mai multe arhitecturi cu memorie partajată.

Modelul cu memorie partajată

- Pe platformele cu memorie pargajată, compilatoarele translatează variabilele din program în adrese de memorie efective care sunt globale.
- Este dificil de implementat un sistem cu memorie distribuită (memoria este fizic distribuită) care să funcționeze ca un sistem cu memorie partajată.

Modelul hibrid

- Două sau mai multe modele anterioare pot fi combinate.
- Un exemplu uzual este combinația dintre modelul cu transmitere de mesaje (MPI) cu modelul cu fire de execuție (PThreads) sau cel cu memorie partajată (OpenMP).
- Un alt exemplu comun este cel care combină modelul cu paralelizarea datelor cu cel cu transmitere de mesaje.
  - Implementările cu paralelizarea datelor pe arhitecturile cu memorie distribuită folosesc modelul cu transmitere de mesaje pentru a transfera date între task-uri în mod transparent pentru programator.

Fire de execuție în C++

- Programarea concurentă a devenit o abilitate esențială pentru toți programatorii C++ deoarece cele mai multe sisteme au mai multe procesoare.
  - Utilizarea eficientă a masinilor multicore necesită abilitatea de codare a aplicațiilor concurente.
- C++ 11 introduce concutența, firele de execuție, variabilele condiționale, mutex, etc.
  - Standardul C++11 se schimba tramatic cu C++ 17: apare STL parallel.
- C++ implementează concurența prin multithreading și paralelism.

Fire de execuție în C++

- C++ multithreading

std::thread thread object care poate fi creat și folosit pentru a rula task-uri în mod independent

- Odată creat, firului ii este transmisa o funcție care să se execute și eventual parametrii pentru acea funcție, care poate fi:
  - Un pointer la o funcție (en., function pointer)
  - Un obiect funcție (en., function object)
  - O expresie lambda
- Deoarece fiecare fir poate executa doar o funcție la un moment dat, putem avea bazine (en., pools) de fire care să ne perimită reutilizarea lor asemănător ca într-un sistem multitasking nelimitat.
  - Astfel, se profită de existența mai multor procesoare și se permite determinarea nivelului la care aplicația doreste să utilizeze resursele sistemului.

Fire de execuție în C++ (comparație cu PThreads)

- Comparativ cu Pthreads, în C++ beneficiem (sau trebuie să avem în vedere) de:
  - Ciclul de viață al obiectelor și tratarea excepțiilor
  - Nu avem o funcție pthread_cancel
  - Clasele pentru mutex, variabile condiție și lacăte (en., locks) folosesc tehnica RAII (*resource acquisition is initialization,* *http://www.stroustrup.com/bs_faq2.html#finally*)
    - RAII reprezintă un aspect principal oferit de biblitecă prin proiectare
  - Avem la dispoziție mecanisme pentru crearea de obiecte apelabile (en., callable objects) și expresii lambda (i.e, funcții anonime) care integrează facilitățile oferite de firele de execuție.

Fire de execuție în C++

```
void print(int n, const std::string &str)  {
   std::cout << „Int: " << n << std::endl;
   std::cout << „String: " << str << std::endl;
}

int main() {
   std::thread t1(print, 5, „One thread");
   t1.join();              //se oprește rularea programului principal până la
                           // terminarea  firului t1
return 0;
}
```

Fire de execuție în C++

- Multithreading

```
std::vector<std::thread> threads;
for (int i = 0; i < s.size(); i++) {
   threads.push_back(std::thread(print, i, s[i]));
}

for (auto &th : threads) {
   th.join();
}
```

Fire de execuție în C++

- ”Funcția firului”: Funcția executată de firul de execuție nu reprezintă ceva special, numai că aceasta nu trebuie să se termine prin aruncarea unei excepții. În această situație se va apela automat std::terminate.
  - https://en.cppreference.com/w/cpp/error/terminate
- Tipuri de lacăte, mutex-uri, strategii
  - std::recursive_mutex, std::timed_mutex; std::unique_lock; std::defer_lock, std::try_to_lock
- Variabile condiție
  - std::future
- Tipuri atomice și funcții pe tipurile atomice
- Referințe *rvalue*, care permit *perfect forwarding* (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2002/n1385.htm) pentru o funcția firului.
- Suport adițional pentru functori (i.e., function objects) prin std::function și std::bind

Fire de execuție în C++: Greșeli frecvente

- Lipsa apelului *join* înainte de terminarea programului determină *crash*. La finalul programului firul va ieși din scop și astfel va fi apelat destructorul, care va apela std::terminate.
- Incercarea de a apela join pe un fir la care s-a apelat detach.
  - Soluția este ca înainte de join sa verificam dacă firul este joinable.
- Trebuie înțeles că apelul std::thread::join() are ca efect **blocarea** firului apelant (adică, a programului principal ... de obicei)!
- Transmiterea parametrilor catre funcția firului nu este implicit prin referință, ci prin valoare!
- Lipsa protejării datelor partajate într-o secțiune critică cu ajutorul *mutex*.
- Lipsa deschiderii lacătelor la ieșirea din secțiunea critică.
- Secțiunile critice trebuie păstrate cât mai mici și compacte.
- Gestiunea proastă a mai multor lacăte: ordinea de închidere/deschidere trebuie atent administrată dacă avem mai multe fire si mai multe lacăte.

Fire de execuție în C++: Greșeli frecvente

- Încercarea de a dobândi std::mutex de două ori poate conduce la comportament nedefinit.
- Nefolosirea std::atomic pentru tipuri de date simple (ex., bool, int). Folosirea mutex în aceste situații conduce la degradarea performanțelor.
- Crearea și distrugerea unui număr mare de fire atunci când avem la dispoziție un *thread pool*.
- Lipsa tratării excepțiilor pentru firele care rulează în background.
- Folosirea firelor pentru simularea task-urilor asincrone. Folosiți std::async.
- Crearea mai multor fire decat numărul de procesoare disponibile.